



中国科学技术大学  
University of Science and Technology of China

# 语言的描述：词法和语法

《编译原理和技术(H)》



# 上下文无关文法

- 正规式与上下文无关文法的比较
- 上下文无关文法的一些基本概念
  - 定义、推导
  - 名词：语言、文法、句型、句子、解析树、语法树



# 正规式的表达能力不足

## □ 正规式的表达能力

- 定义一些简单的语言，能表示给定结构的固定次数的重复或者没有指定次数的重复

例：  $a(ba)^5, a(ba)^*$

- 不能描述配对或嵌套的结构

□ 例1：配对括号串的集合，如不能表达  $(^n)^n, n \geq 1$

原因：  $n$  不固定，无法表示右括号的个数必须正好与前面左括号的个数一样

□ 例2：  $\{wcw \mid w \text{ 是由 } a \text{ 和 } b \text{ 组成的串}\}$

原因：  $w$  的长度不固定，  $c$  后面的串要依据  $c$  前面不定长的串  $w$  来确定；

有限自动机有有限个状态，无法记录访问同一状态的次数

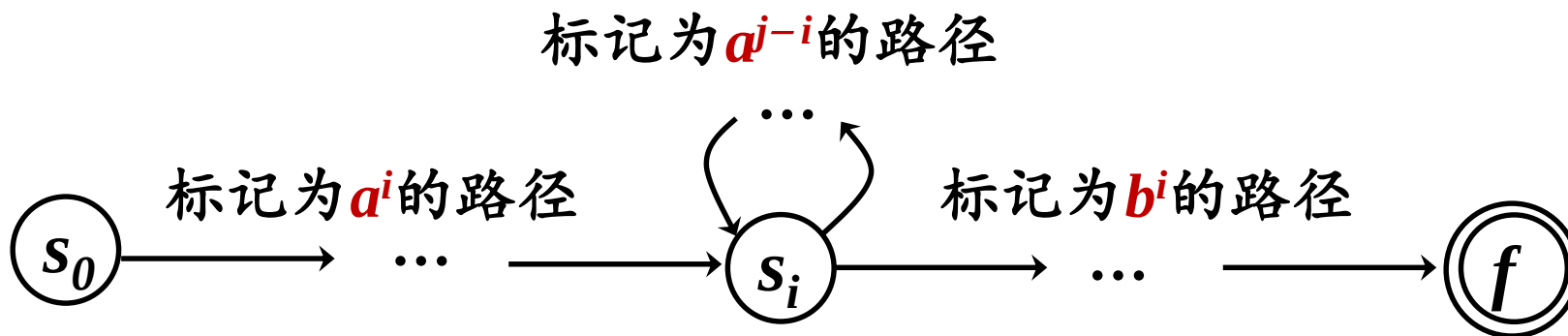


# 正规式的表达能力不足

例：  $L = \{a^n b^n \mid n \geq 1\}$ ， $L$  不能用正规式描述

## 反证法

- 若存在接受  $L$  的 DFA  $D$ ，状态数为  $k$  个（有限个）
- 设  $D$  读完  $\varepsilon, a, aa, \dots, a^k$  分别到达状态  $s_0, s_1, \dots, s_k$
- 至少有两个状态相同，例如是  $s_i$  和  $s_j$ ，则  $a^j b^i$  属于  $L$ ，这与假设相矛盾





# 上下文无关文法的定义

Context-free **Grammar** (CFG)

注: **Syntax**-语法

## □ CFG是四元组 $(V_T, V_N, S, P)$

$V_T$ : 终结符(terminal, 记号名, 即token的第1元)集合

$V_N$ : 非终结符(nonterminal)集合

$S$ : 开始符号(start symbol), 是一个非终结符

$P$ : 产生式(production)集合

产生式的形式:  $A \rightarrow \alpha$ ,  $A \in V_N, \alpha \in (V_T \cup V_N)^*$

有时用  $A ::= \alpha$

■ 例 (  $\{\text{id}, +, *, -, (, )\}, \{\text{expr}, \text{op}\}, \text{expr}, P$  ) 第一个产生式左部的符号就是文法开始符号

$\text{expr} \rightarrow \text{expr op expr}$

$\text{expr} \rightarrow (\text{expr})$

$\text{expr} \rightarrow - \text{expr}$

$\text{expr} \rightarrow \text{id}$

$\text{op} \rightarrow +$

$\text{op} \rightarrow *$



# CFG的简化表示

## □ 表达式的CFG的简化表示

### ■ 引入选择符 |

$expr \rightarrow expr\ op\ expr \mid (expr) \mid -expr \mid id$

$op \rightarrow + \mid *$

注：+, \*是 $op$ 的选择(alternatives)

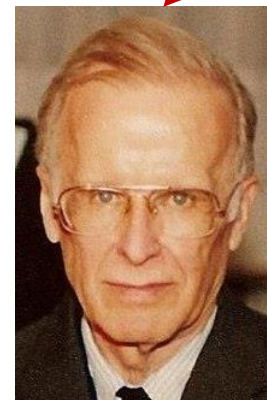
### ■ 简化名称

$E \rightarrow E\ A\ E \mid (E) \mid -E \mid id$

$A \rightarrow + \mid *$

Backus-Naur Form

巴科斯-诺尔, 范式



John Backus  
1977图灵奖  
首次在ALGOL 58中实现BNF



Peter Naur  
2005图灵奖  
在ALGOL 60中发展和简化



## □ John Backus (1924-2007)

### ■ 1977图灵奖获奖成就

- Fortran发明组组长
- 提出了BNF

### ■ 履历

- 弗吉尼亚大学化学(因出勤率低而开除, 随后入伍)
- 哥伦比亚大学数学BS 1949, AM 1950
- IBM

### ■ 获奖演说: [Can Programming Be Liberated From the von Neumann Style?](#)

## □ Peter Naur (1928-2016)

### ■ 2005图灵奖获奖成就

- ALGOL 60
- 发展BNF并简化

### ■ 履历

- 哥本哈根大学天文学BS 1949、博士 1957
- 1950-51剑桥: 天气恶劣破坏天文观测计划, 但花很多时间编程以解决天文学中的扰动问题
- 毕业后转向计算机科学
- 获奖演说: [Computing vs. Human Thinking](#)



# 正规式和CFG的比较

- 都能表示语言
- 凡是能用正规式表示的语言，都能用CFG表示

- 正规式

$(a|b)^*ab$

- 上下文无关文法CFG

可机械地由NFA变换而得，NFA的字母表视为终结符集合

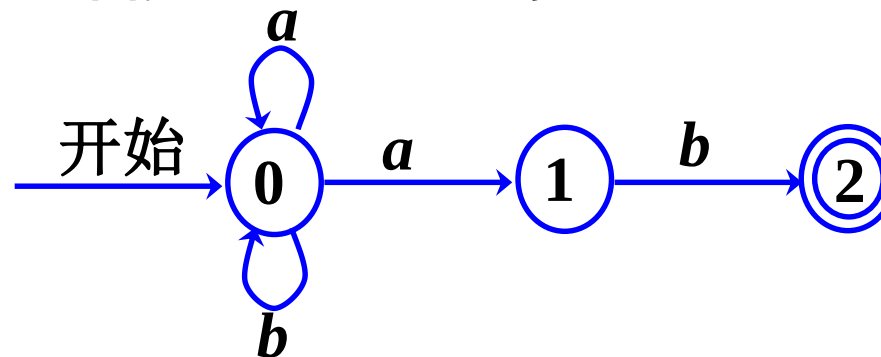
为每个NFA状态  $i$  引入一个非终结符  $A_i$ ，NFA中每条弧对应于产生式的一个分支（选择），

对于接受状态  $i$ ，则引入  $A_i \rightarrow \varepsilon$

$A_0 \rightarrow a A_0 \mid b A_0 \mid a A_1$

$A_1 \rightarrow b A_2$

$A_2 \rightarrow \varepsilon$  (该产生式并不必要)



如果状态  $i$  有一个  $a$  转换到状态  $j$ ，则引入产生式  $A_i \rightarrow a A_j$ ；  
如果是  $\varepsilon$  转换，则引入  $A_i \rightarrow A_j$ 。



# 分离词法分析器的理由

- 正规文法是上下文无关文法的特例
- 为什么要用正规式定义词法
  - 词法规则非常简单，正规式描述简洁且易于理解
  - 从正规式构造出的词法分析器（DFA）效率高
- 分离词法分析和语法分析的好处 (从软件工程看)
  - 简化设计，便于编译器前端的模块划分
  - 改进编译器的效率，增强编译器的可移植性，例如将输入字符集的特殊性限制在词法分析时处理
- 分离但在同一遍（Pass）中进行
  - 是通常编译器的做法



# 上下文无关文法

- 正规式与上下文无关文法的比较
- 上下文无关文法的一些基本概念
  - 定义、推导
  - 名词：语言、文法、句型、句子、解析树、语法树



# CFG: 推导

## □ 推导(derivation)

- 是从文法推出文法所描述的语言中**合法串**集合的动作
- 把产生式看成重写规则，把符号串中的非终结符用其产生式右部的串来代替

例  $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$

$E \Rightarrow -E$  读作  $E$  推导出  $-E$

$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\text{id} + E) \Rightarrow -(\text{id} + \text{id})$

上述代换序列称为从  $E$  到  $-(\text{id}+\text{id})$  的**推导**       $-(\text{id}+\text{id})$  是  $E$  的**实例**

## 记法

- 如果  $A \rightarrow \gamma$  是产生式， $\alpha$  和  $\beta$  是文法的任意符号串，那么可以说  $\alpha A \beta \Rightarrow \alpha \gamma \beta$
- 0步或多步推导  $S \Rightarrow^* \alpha$ : 1)  $\forall \alpha. \alpha \Rightarrow^* \alpha$ ; 2) 如果  $\alpha \Rightarrow^* \beta$ ,  $\beta \Rightarrow \gamma$ , 那么  $\alpha \Rightarrow^* \gamma$
- 一步或多步推导  $S \Rightarrow^+ w$



# 思考

□ 上下文无关是什么意思？



## □ 上下文无关是什么意思？

■ 指对于文法推导的每一步  $\alpha A \beta \Rightarrow \alpha \gamma \beta$

文法符号串  $\gamma$  仅依据  $A$  的产生式推导，不依赖  $A$  的上下文  $\alpha$  和  $\beta$



# 语言、文法、句型、句子

## □ 上下文无关语言

- 由上下文无关文法 $G$ 产生的语言称为上下文无关语言，它包含：  
从开始符号 $S$ 出发，经 $\Rightarrow^+$ 推导所能到达的所有仅由终结符组成的串
- 句型(sentential form):  $S \Rightarrow^* \alpha$ ,  $S$ 是开始符号,  $\alpha$ 是由终结符和/或非终结符组成的串, 则 $\alpha$ 是文法 $G$ 的句型
- 句子(sentence):  $S \Rightarrow^+ w$ ,  $w$ 是仅由终结符组成的句型

## □ 等价的文法

- 它们产生同样的语言



# 最左推导与最右推导

例  $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$

## □ 最左推导(leftmost derivation)

每步代换句型中**最左边**的非终结符

$$\begin{aligned} E &\Rightarrow_{lm} -E \Rightarrow_{lm} -(E) \Rightarrow_{lm} -(E + E) \\ &\Rightarrow_{lm} -(\text{id} + E) \Rightarrow_{lm} -(\text{id} + \text{id}) \end{aligned}$$

## □ 最右推导 (rightmost or canonical, 规范推导)

每步代换句型中**最右边**的非终结符

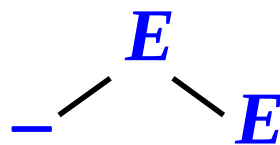
$$\begin{aligned} E &\Rightarrow_{rm} -E \Rightarrow_{rm} -(E) \Rightarrow_{rm} -(E + E) \\ &\Rightarrow_{rm} -(E + \text{id}) \Rightarrow_{rm} -(\text{id} + \text{id}) \end{aligned}$$



# 解析树(parse tree)

例  $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$

-(id+id)最左推导的解析树 ( parse tree )



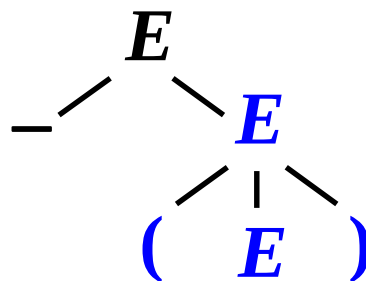
$$E \Rightarrow -E$$



# 解析树(parse tree)

例  $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$

-(id+id)最左推导的解析树 (parse tree)



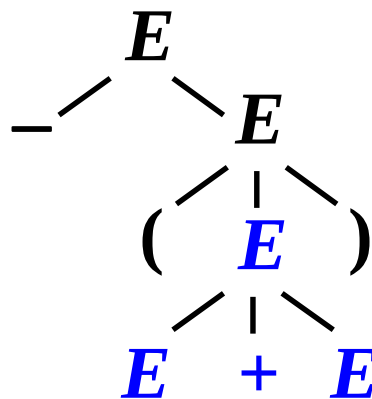
$E \Rightarrow -E \Rightarrow -(E)$



# 解析树(parse tree)

例  $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$

-(id+id)最左推导的解析树 ( parse tree )



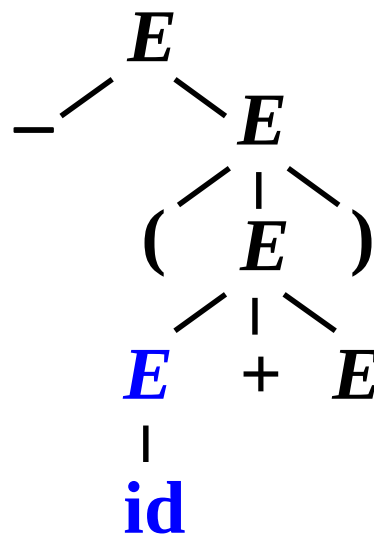
$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E)$



# 解析树(parse tree)

例  $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$

**-(id+id)**最左推导的解析树 ( **parse tree** )



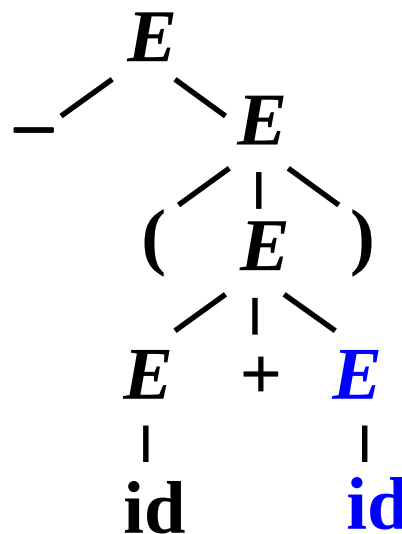
$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\text{id} + E)$



# 解析树(parse tree)

例  $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$

**-(id+id)**最左推导的解析树 ( **parse tree** ) ——解析树的结构与文法有关



$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\text{id} + E) \Rightarrow -(\text{id} + \text{id})$



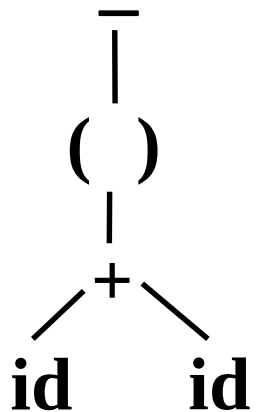
# 语法树(syntax tree)

## □ 语法树是解析树的浓缩表示

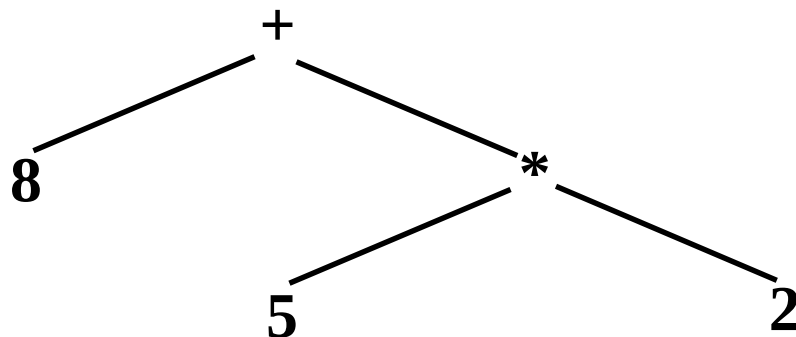
每个结点表示一个语法构造，算符和关键字是语法树中的内部结点

举例：

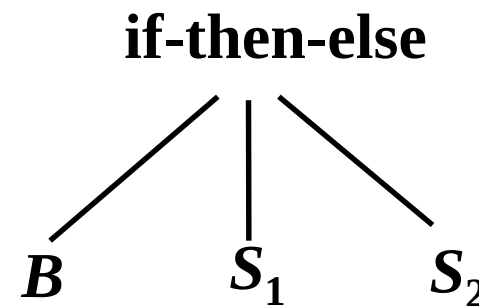
-(id+id)



8 + 5 \* 2



if B then S<sub>1</sub> else S<sub>2</sub>



语法制导翻译可以基于解析树，也可以基于语法树



中国科学技术大学  
University of Science and Technology of China

# 语言 and 文法

- 语言和文法：验证、消除二义性
- 非上下文无关文法



# 验证文法产生的语言

文法  $G: S \rightarrow '(S)' S \mid \varepsilon$       语言  $L(G) =$  配对的括号串的集合

- ① 从  $S$  推出的是配对的括号串
- ② 任意配对的括号串可由  $S$  推出

## □ 证明①：按推导步数进行归纳

按任意步推导，推出的是配对括号串

- 归纳基础(Basis):  $S \Rightarrow \varepsilon$
- 归纳 (Induction) 假设  
少于  $n$  步的推导都产生配对的括号串, 如  $S \Rightarrow^* x, S \Rightarrow^* y$
- 归纳步骤:  $n$  步的最左推导如下:

$$S \Rightarrow '(S)' S \Rightarrow^* '(' x ')' S \Rightarrow^* '(' x ')' y$$



# 验证文法产生的语言

文法  $G : S \rightarrow '(S \text{ ')}' S \mid \varepsilon$     语言  $L(G) =$  配对的括号串的集合

□ 证明②：任意长度的配对括号串均可由  $S$  推导出来

按串长进行归纳

- 归纳基础(Basis):  $S \Rightarrow \varepsilon$
- 归纳 (Induction)假设: 长度小于  $2n$  的配对的括号串都可以从  $S$  推导出来
- 归纳步骤: 考虑长度为  $2n(n \geq 1)$  的  $w = '(x \text{ ')}' y$

$$S \Rightarrow '(S \text{ ')}' S \Rightarrow^* '(x \text{ ')}' S \Rightarrow^* '(x \text{ ')}' y$$



# 文法的二义性

文法的某些句子存在不止一种最左(最右)推导, 或者不止一棵解析树, 则该文法是**二义**的。

例  $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$

**id\*id+id** 有两个不同的最左推导

$$E \Rightarrow E * E$$

$$\Rightarrow \text{id} * E$$

$$\Rightarrow \text{id} * E + E$$

$$\Rightarrow \text{id} * \text{id} + E$$

$$\Rightarrow \text{id} * \text{id} + \text{id}$$

$$E \Rightarrow E + E$$

$$\Rightarrow E * E + E$$

$$\Rightarrow \text{id} * E + E$$

$$\Rightarrow \text{id} * \text{id} + E$$

$$\Rightarrow \text{id} * \text{id} + \text{id}$$



# 文法的二义性

**id\*id+id** 有两棵不同的解析树

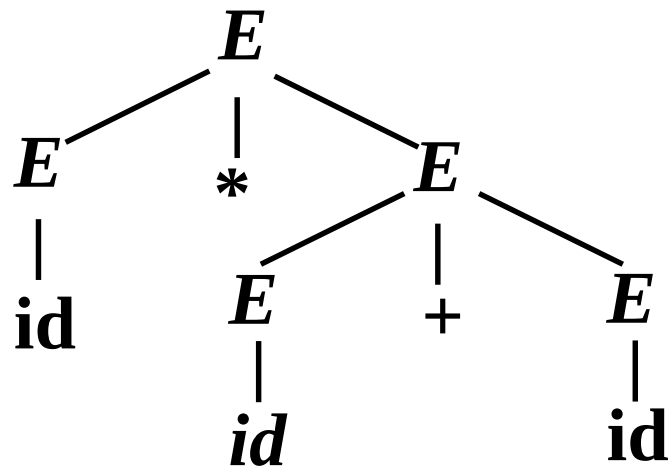
$E \Rightarrow E * E$

$\Rightarrow id * E$

$\Rightarrow id * E + E$

$\Rightarrow id * id + E$

$\Rightarrow id * id + id$



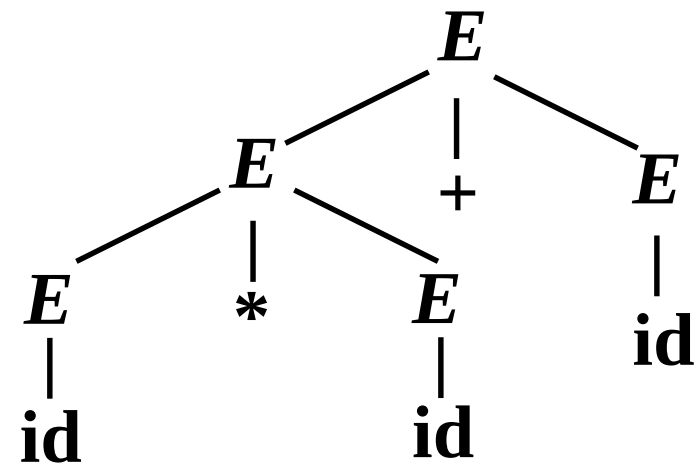
$E \Rightarrow E + E$

$\Rightarrow E * E + E$

$\Rightarrow id * E + E$

$\Rightarrow id * id + E$

$\Rightarrow id * id + id$





# 表达式的另一种文法

## □ 用一种层次的观点看待表达式

id \* id \* (id+id) + id \* id + id

对于一个语言L,  
可以有多个**文法(grammar)**  
来表达其**语法(syntax)**

## □ 无二义的文法

$expr \rightarrow \textcolor{red}{expr} + term \mid term$

$term \rightarrow \textcolor{red}{term} * factor \mid factor$

$factor \rightarrow id \mid (expr)$

+ 是自左向右结合

如果改成

$expr \rightarrow term + \textcolor{red}{expr} \mid term$   
呢?

+ 是自右向左结合

\* 是自左向右结合

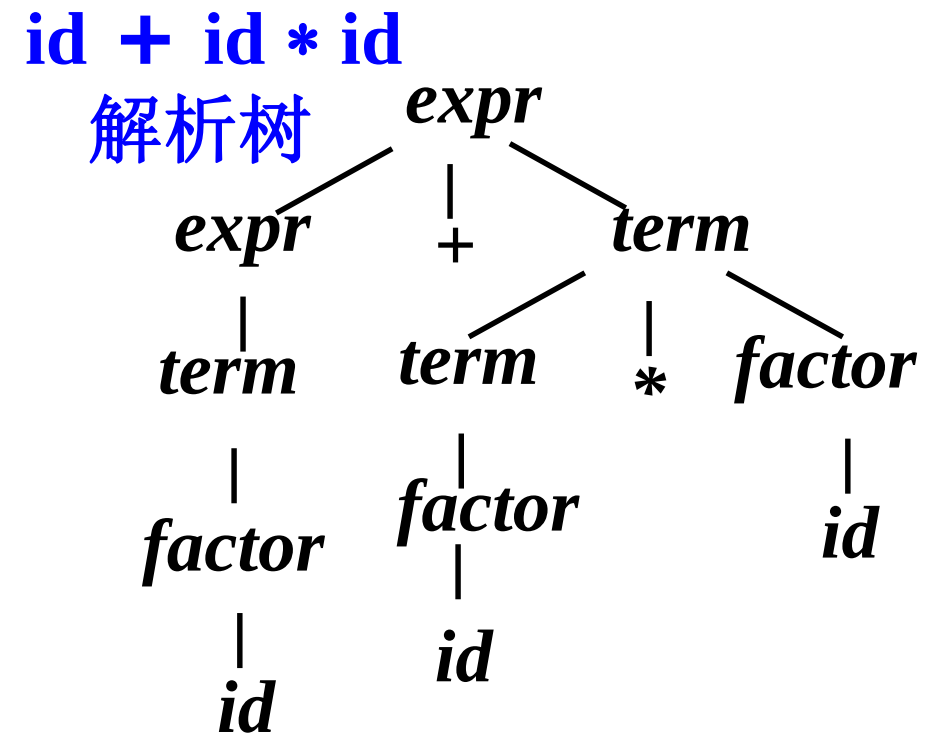
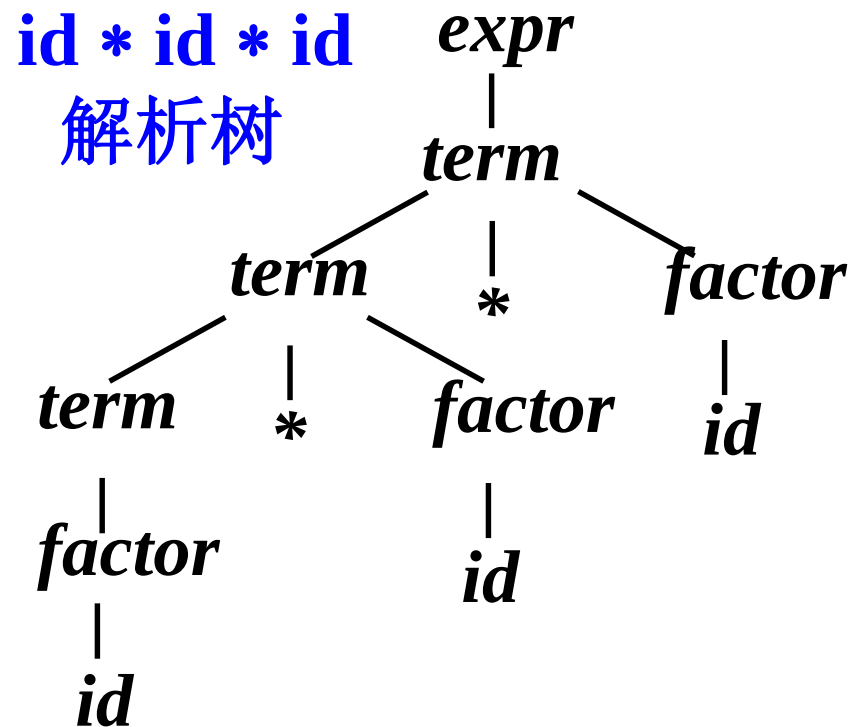


# 表达式的另一种文法

$expr \rightarrow expr + term \mid term$

$term \rightarrow term * factor \mid factor$

$factor \rightarrow id \mid (expr)$





# 消除二义性(Eliminating ambiguity)

$stmt \rightarrow$  if  $expr$  then  $stmt$   
          | if  $expr$  then  $stmt$  else  $stmt$   
          | other

□ 句型: if  $expr$  then if  $expr$  then  $stmt$  else  $stmt$

有两个最左推导:

$stmt \Rightarrow$  if  $expr$  then  $stmt$   
           $\Rightarrow$  if  $expr$  then if  $expr$  then  $stmt$  else  $stmt$

$stmt \Rightarrow$  if  $expr$  then  $stmt$  else  $stmt$   
           $\Rightarrow$  if  $expr$  then if  $expr$  then  $stmt$  else  $stmt$



# 消除二义性

## □ 无二义的文法

$stmt \rightarrow matched\_stmt$   
 $\quad | unmatched\_stmt$

else 的就近匹配规则

$matched\_stmt \rightarrow if\ expr\ then\ matched\_stmt$   
 $\quad \quad \quad else\ matched\_stmt$   
 $\quad | other$

$unmatched\_stmt \rightarrow if\ expr\ then\ stmt$   
 $\quad | if\ expr\ then\ matched\_stmt$   
 $\quad \quad \quad else\ unmatched\_stmt$



中国科学技术大学  
University of Science and Technology of China

# 语言和文法

- 语言和文法：验证、消除二义性
- 非上下文无关文法



# 形式语言鸟瞰

文法  $G = (V_T, V_N, S, P)$

□ 0型文法:  $\alpha \rightarrow \beta$ ,  $\alpha, \beta \in (V_N \cup V_T)^*$ ,  $|\alpha| \geq 1$

短语文法

□ 1型文法:  $|\alpha| \leq |\beta|$ , 但  $S \rightarrow \varepsilon$  可以例外

上下文有关文法

□ 2型文法:  $A \rightarrow \beta$ ,  $A \in V_N$ ,  $\beta \in (V_N \cup V_T)^*$

上下文无关文法

□ 3型文法:  $A \rightarrow aB$  或  $A \rightarrow a$  或  $A \rightarrow \varepsilon$ ,  $A, B \in V_N$ ,  $a \in V_T$

正规文法



# 上下文无关文法的优缺点

## □ 优点

- 文法给出了精确的、易于理解的语法说明
- 可以给语言定义出层次结构
- 可以基于文法自动产生高效的分析器
- 以文法为基础实现语言便于对语言修改

## □ 缺点

- 表达能力不足够，只能描述编程语言中的大部分语法



# 非上下文无关的语言构造

$$L_1 = \{wcw \mid w \text{属于}(a \mid b)^*\}$$

用来抽象：标识符的声明应先于其引用

C、Java都不是上下文无关语言

$$L_2 = \{a^n b^m c^n d^m \mid n \geq 0, m \geq 0\}$$

用来抽象：形参个数和实参个数应该相同

$$L_3 = \{a^n b^n c^n \mid n \geq 0\}$$

用来抽象：早先排版描述的一个现象

b e g i n: 5个字母键, 5个回退键, 5个下划线键



# 形似的上下文无关语言

$wcw$

$$L_1' = \{wcw^R \mid w \in (a|b)^*\}$$

$$S \rightarrow aSa \mid bSb \mid c$$

$a^n b^m c^n d^m$

$$L_2' = \{a^n b^m c^m d^n \mid n \geq 1, m \geq 1\}$$

$$S \rightarrow aSd \mid aAd$$

$$A \rightarrow bAc \mid bc$$

$a^n b^n c^n$

$$L_2'' = \{a^n b^n c^m d^m \mid n \geq 1, m \geq 1\}$$

$$S \rightarrow AB$$

$$A \rightarrow aAb \mid ab$$

$$B \rightarrow cBd \mid cd$$



# 上下文有关文法

例  $L_3 = \{a^n b^n c^n \mid n \geq 1\}$  的上下文有关文法

$S \rightarrow aSBC$      $S \rightarrow aBC$      $CB \rightarrow BC$      $aB \rightarrow ab$

$bB \rightarrow bb$      $bC \rightarrow bc$      $cC \rightarrow cc$

$a^n b^n c^n$  的推导过程如下:

$S \Rightarrow^* a^{n-1} S(BC)^{n-1}$     用  $S \rightarrow aSBC$   $n-1$  次

$S \Rightarrow^+ a^n (BC)^n$     用  $S \rightarrow aBC$  1 次

$S \Rightarrow^+ a^n B^n C^n$     用  $CB \rightarrow BC$  交换相邻的  $CB$

$S \Rightarrow^+ a^n b B^{n-1} C^n$     用  $aB \rightarrow ab$  1 次

$S \Rightarrow^+ a^n b^n C^n$     用  $bB \rightarrow bb$   $n-1$  次

$S \Rightarrow^+ a^n b^n c C^{n-1}$     用  $bC \rightarrow bc$  1 次

$S \Rightarrow^+ a^n b^n c^n$     用  $cC \rightarrow cc$   $n-1$  次



## 例题3-1 写等价的非二义文法

下面的二义文法描述命题演算公式的语法，为它写一个等价的非二义文法

$$S \rightarrow S \text{ and } S \mid S \text{ or } S \mid \text{not } S \mid p \mid q \mid '(' S ')'$$

解答

非二义文法的产生式如下：

$$E \rightarrow E \text{ or } T \mid T$$

$$T \rightarrow T \text{ and } F \mid F$$

$$F \rightarrow \text{not } F \mid '(' E ') ' \mid p \mid q$$



## 例题3-1 写等价的非二义文法

下面的二义文法描述命题演算公式的语法，为它写一个等价的非二义文法

$$S \rightarrow S \text{ and } S \mid S \text{ or } S \mid \text{not } S \mid p \mid q \mid '(' S ')'$$

解答

非二义文法的产生式如下：

$$E \rightarrow E \text{ or } T \mid T$$

$$T \rightarrow T \text{ and } F \mid F$$

$$F \rightarrow \text{not } E \mid '(' E ') \mid p \mid q \quad ?$$

not p and q有两种不同的最左推导

not p and q

not p and q



## 例题3-2 写等价的不同文法

设计一个文法：字母表 $\{a, b\}$ 上  $a$  和  $b$  的个数相等的所有串的集合

□ 二义文法： $S \rightarrow a S b S \mid b S a S \mid \varepsilon$   
 $aabbabab$   $aabbabab$

□ 二义文法： $S \rightarrow a B \mid b A \mid \varepsilon$   
 $A \rightarrow a S \mid b A A$   
 $B \rightarrow b S \mid a B B$   
 $aabbabab$   $aabbabab$   $aabbabab$

□ 非二义文法： $S \rightarrow a B S \mid b A S \mid \varepsilon$   
 $A \rightarrow a \mid b A A$   
 $B \rightarrow b \mid a B B$   
 $a a b b a b a b$   
 $a B S$



中国科学技术大学  
University of Science and Technology of China

# 自上而下解析的一般方法



# 自上而下解析的一般方法

## □ 自上而下 (top-down) 解析

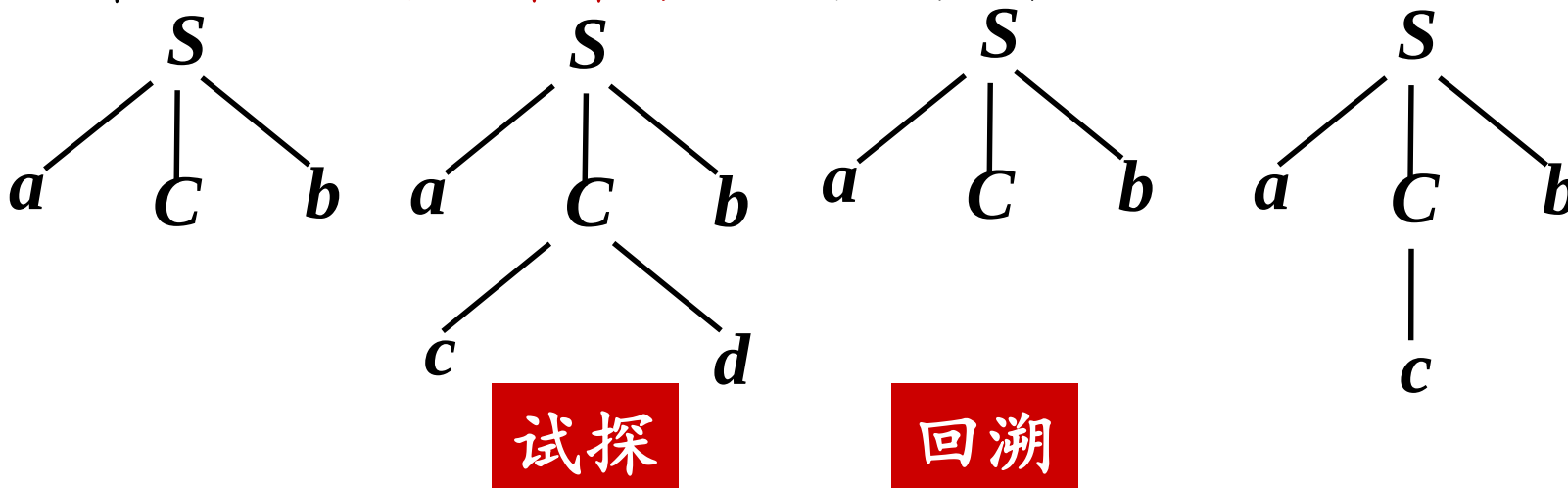
从开始符号出发, 为输入串寻找**最左推导**

**试探** 产生式的选择 – 失败**回溯**(效率低, 代价高)

■ ANTLR: 引入**带谓词的DFA**使得**回溯**后不重新分析输入串

例 文法  $S \rightarrow aCb$      $C \rightarrow cd \mid c$

为输入串  $w = acb$  **建立解析树** (从根结点开始)





# 自上而下解析：左递归

## □ 文法左递归

- 若文法中存在某非终结符 $A$ ，使得 $A \Rightarrow^+ A\alpha$ ，其中 $\alpha$ 是文法符号串

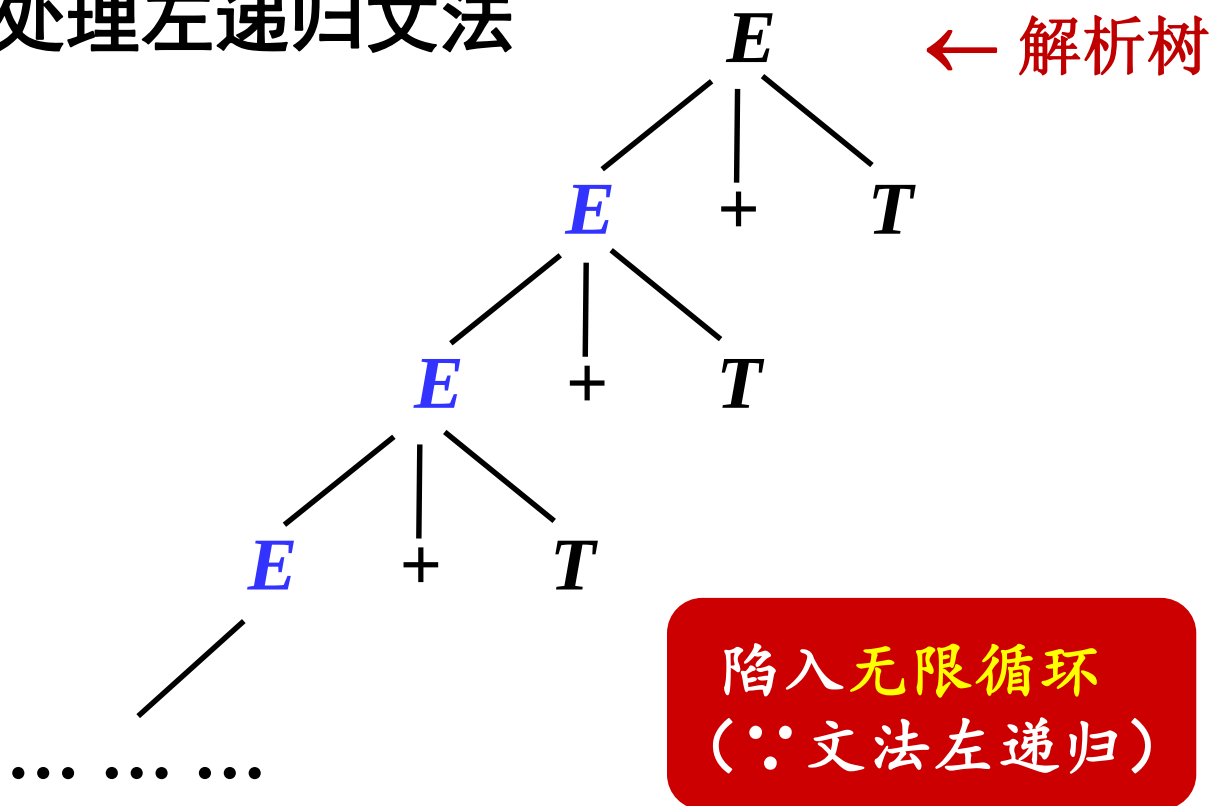
## □ 自上而下解析不能处理左递归文法

算术表达式文法

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow ( E ) \mid \text{id}$$





# 消除左递归(Eliminating left recursion)

## □ 文法左递归

- 若文法中存在某非终结符 $A$ , 使得 $A \Rightarrow^+ A\alpha$ , 其中 $\alpha$ 是文法符号串

## □ 直接左递归 (immediate left recursion)

若文法中存在  $A \rightarrow A\alpha \mid \beta$ , 其中文法符号串 $\beta$ 不以 $A$ 开头

- 由 $A$ 推出的串的特点  $A \Rightarrow^+ \beta\alpha \dots \alpha$

## □ 消除直接左递归

文法  $A \rightarrow A\alpha \mid \beta$ , 可写成  $A \rightarrow \beta A'$

$$A' \rightarrow \alpha A' \mid \varepsilon$$



# 消除左递归

## 例 算术表达式文法

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow ( E ) \mid \text{id}$$

## 消除左递归后的文法

$$E \rightarrow TE'$$

$$E' \rightarrow + TE' \mid \varepsilon$$

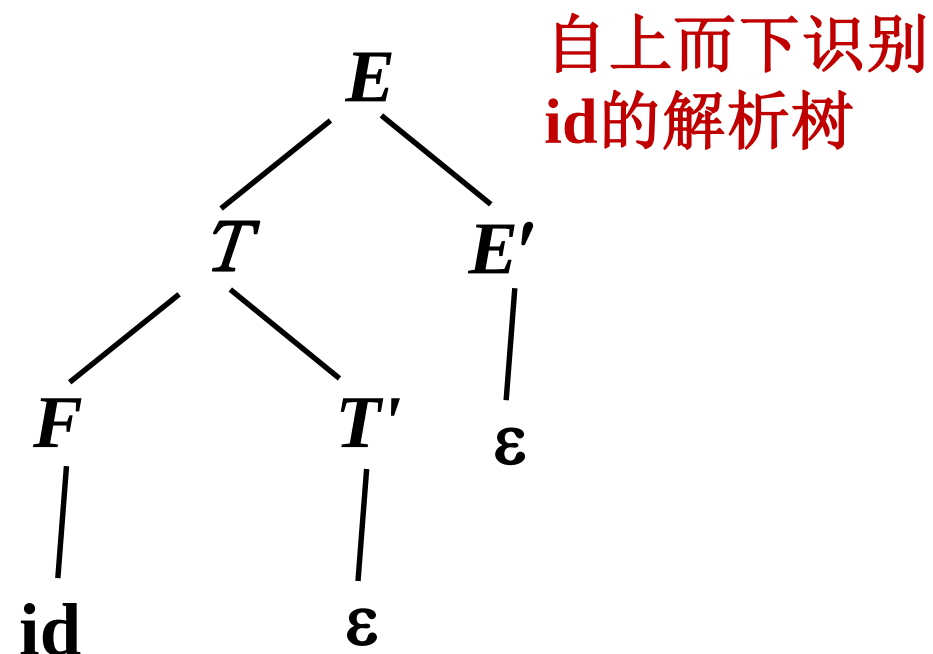
$$T \rightarrow FT'$$

$$T' \rightarrow * FT' \mid \varepsilon$$

$$F \rightarrow ( E ) \mid \text{id}$$

$$E \Rightarrow^+ T + T \dots + T$$

$$T \Rightarrow^+ F * F \dots * F$$





# 消除非直接左递归

## □ 间接左递归

$$S \rightarrow Aa \mid b$$

$$A \rightarrow Sd \mid \varepsilon$$

## □ 先变换成直接左递归

$$S \rightarrow Aa \mid b$$

$$A \rightarrow Aa \mid b \mid \varepsilon$$

## □ 再消除左递归

$$S \rightarrow Aa \mid b$$

$$A \rightarrow baA' \mid A'$$

$$A' \rightarrow aaA' \mid \varepsilon$$

## □ 隐藏左递归

$$\Sigma \rightarrow A \mathbf{B} \chi$$

$$\mathbf{A} \rightarrow \varepsilon$$

$$B \rightarrow A \mathbf{B} \varepsilon \mid X \delta$$

$$X \rightarrow \Sigma \phi$$

$B \Rightarrow ABe \Rightarrow Be$  是隐藏直接左递归，  
而  $S \Rightarrow ABc \Rightarrow Bc \Rightarrow Cdc \Rightarrow Sfcd$  是  
隐藏间接左递归。

ANTLR4 接受的文法包含  
直接左递归，但不包含间  
接或隐藏的左递归



# 提左因子(left factoring)

□ 有左因子的(left-factored)文法:  $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$

■ 自上而下解析时, 不清楚应该用A的哪个选择来代换

□ 提左因子

$$A \rightarrow \alpha A' \qquad A' \rightarrow \beta_1 \mid \beta_2$$

例 悬空else的文法

$$\begin{aligned} stmt \rightarrow & \text{if } expr \text{ then } stmt \text{ else } stmt \\ & \mid \text{if } expr \text{ then } stmt \mid \text{other} \end{aligned}$$

提左因子

$$\begin{aligned} stmt \rightarrow & \text{if } expr \text{ then } stmt \text{ optional\_else\_part} \mid \text{other} \\ \text{optional\_else\_part} \rightarrow & \text{else } stmt \mid \varepsilon \end{aligned}$$



# 自下而上解析

(移进-归约解析)

- 归约(最右推导的逆过程)
- 句柄(可归约串),可能不唯一
- 冲突: 移进-归约、归约-归约



# 归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例  $S \rightarrow aABe$   
 $A \rightarrow Abc \mid b$   
 $B \rightarrow d$

输入串:  $abbcde$

**一步归约**：将与某个产生式右部匹配  
的子串代换成该产生式的左部符号



如  $A \rightarrow b$



# 归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例  $S \rightarrow aABe$

$A \rightarrow Abc \mid b$

$B \rightarrow d$

输入串:  $abbcde$

$ab$  (读入 $ab$ ) 寻找能匹配某产生式右部的子串

$A \rightarrow b$

$$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$$

**右句型:** 按最右推导推出的句型,  $aABe$ 、 $aAde$ 、 $aAbcde$ 、 $abbcde$  都是右句型



# 归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

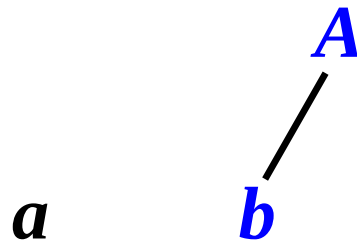
例  $S \rightarrow aABe$   
 $A \rightarrow Abc \mid b$   
 $B \rightarrow d$

输入串:  $abbcde$

$ab$

$aA$  (归约)

用产生式  $A \rightarrow b$   
归约后仍能形成右句型  
 $aAbcde$  是右句型



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$

**右句型:** 按最右推导推出的句型,  $aABe$ 、 $aAde$ 、 $aAbcde$ 、 $abbcde$  都是右句型



# 归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

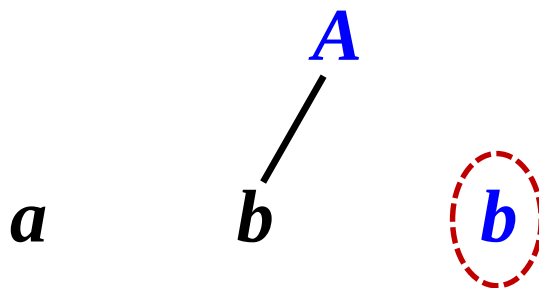
例  $S \rightarrow aABe$   
 $A \rightarrow Abc \mid b$   
 $B \rightarrow d$

输入串:  $abbcde$

$ab$

$aAb$  (再读入  $b$ )

$A \rightarrow b$   
 $b$  可以归约成  $A$  吗?



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$



# 归约 (Reduce)

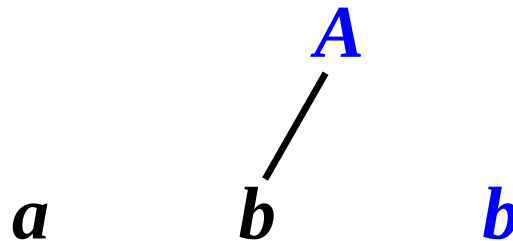
把输入串归约成文法的开始符号，是最右推导的逆过程

例  $S \rightarrow aABe$   
 $A \rightarrow Abc \mid b$   
 $B \rightarrow d$

输入串:  $abbcde$

$ab$

$aAb$  (再读入  $b$ )



$A \rightarrow b$   
 $b$  可以归约成  $A$  吗?

归约后能否形成右句型?  
 $aAAcde$  不是右句型  
故不能将  $b$  归约成  $A$

$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$



# 归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例  $S \rightarrow aABe$

$A \rightarrow \textcolor{blue}{Abc} \mid b$

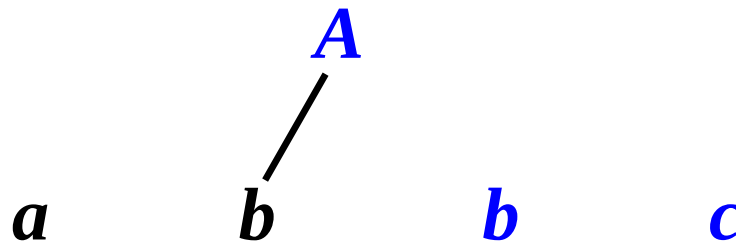
$B \rightarrow d$

$A \rightarrow \textcolor{blue}{Abc}$

输入串:  $abbcde$

$\textcolor{blue}{ab}$

$\textcolor{blue}{aAbc}$  (再读入  $c$ )



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} \textcolor{blue}{aAbc}de \Rightarrow_{rm} abbcde$



# 归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例  $S \rightarrow aABe$

$A \rightarrow \textcolor{blue}{Abc} \mid b$

$B \rightarrow d$

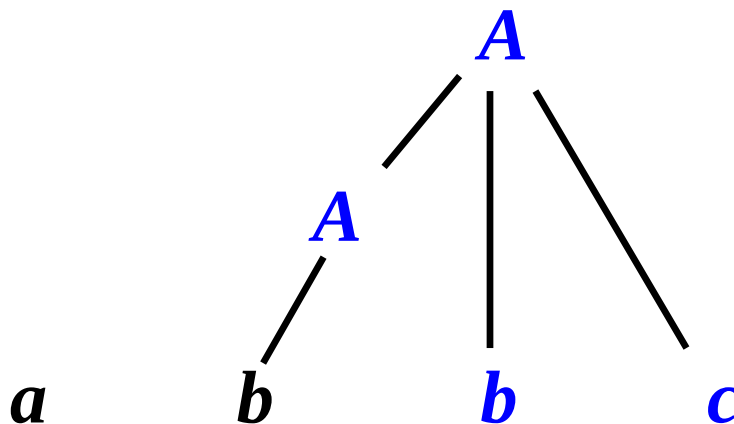
$A \rightarrow \textcolor{blue}{Abc}$

输入串:  $abbcde$

$ab$

$a\textcolor{blue}{Abc}$

$a\textcolor{blue}{A}$  (归约)



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} a\textcolor{blue}{A}de \Rightarrow_{rm} a\textcolor{blue}{Abc}de \Rightarrow_{rm} abbcde$



# 归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例  $S \rightarrow aABe$

$A \rightarrow Abc \mid b$

$B \rightarrow d$

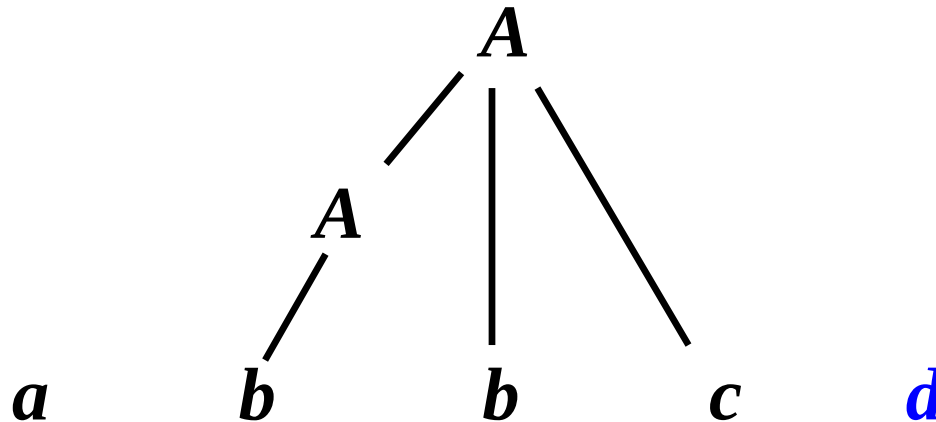
$B \rightarrow d$

输入串:  $abbcde$

$ab$

$aAbc$

$aAd$  (再读入  $d$ )



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$



# 归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例  $S \rightarrow aABe$

$A \rightarrow Abc \mid b$

$B \rightarrow d$

$B \rightarrow d$

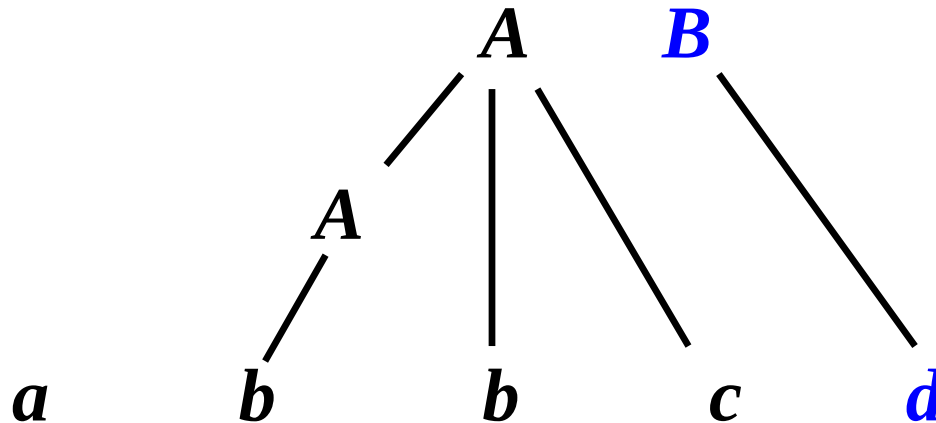
输入串:  $abbcde$

$ab$

$aAbc$

$aAd$

$aAB$  (归约)



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$



# 归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例  $S \rightarrow aABe$

$A \rightarrow Abc \mid b$

$B \rightarrow d$

输入串: *abbcd**e*

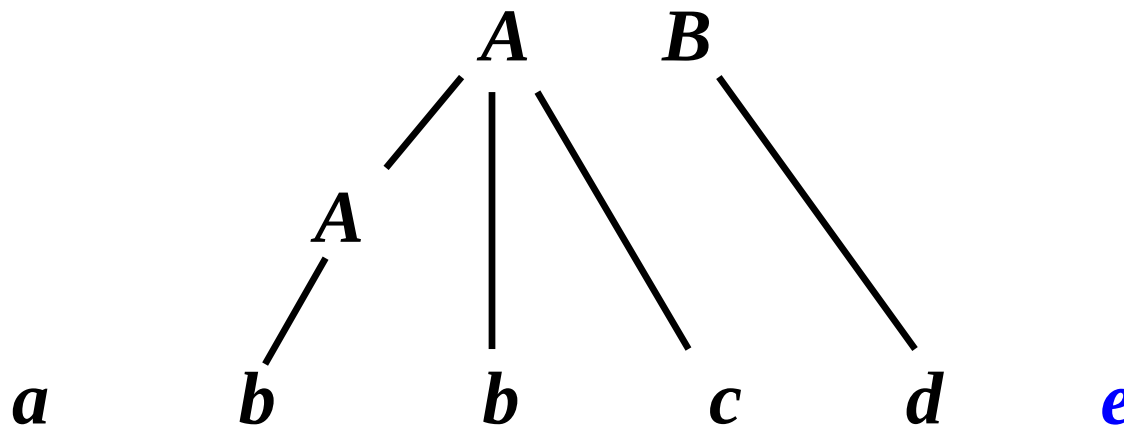
*a**b*

*a**A**bc*

*a**A**d*

*a**A**B*

*a**A**B**e*(再读入*e*)



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} a\underline{Abc}de \Rightarrow_{rm} abbcde$



# 归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例  $S \rightarrow aABe$

$A \rightarrow Abc \mid b$

$B \rightarrow d$

输入串:  $abbcd e$

$ab$

$aAbc$

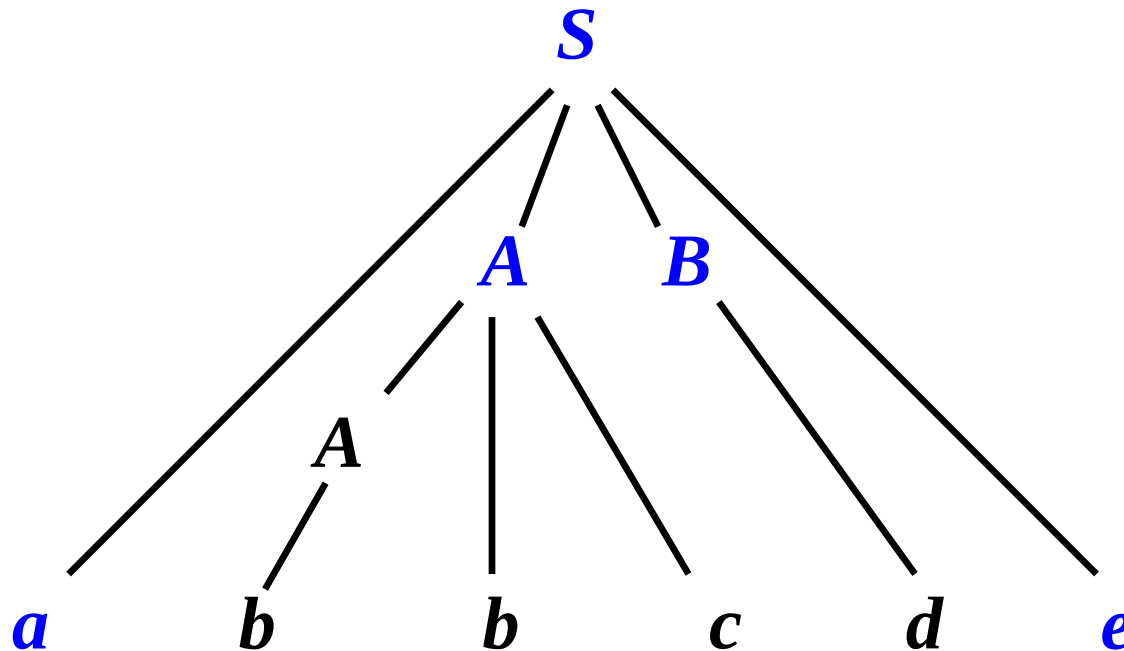
$aAd$

$aAB$

$aABe$

$S$  (归约)

$S \rightarrow aABe$



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$



## 3.5 自下而上解析

(移进-归约解析)

- 归约(最右推导的逆过程)
- 句柄(可归约串),可能不唯一
- 冲突: 移进-归约、归约-归约



# 句柄(handles)

## □ 右句型的句柄（可归约串）

■ 是右句型  $\alpha\beta w$  中和某产生式  $A \rightarrow \beta$  右部匹配的最左子串  $\delta$ , 并且

■ 把  $\beta$  归约成  $A$  代表了最右推导的逆过程的一步

右句型  $\alpha\beta w$  中将子串  $\beta$  归约成  $A$  后得到的串  $\alpha Aw$  仍是右句型

$S \rightarrow aABe$

\* 右句型：最右推导可得的句型

$A \rightarrow Abc \mid b$

$B \rightarrow d$

$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$

■ 句柄的右边仅含终结符（是尚未处理输入串）

■ 如果文法二义，那么句柄可能不唯一



# 例句柄不唯一

$$E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$$

$$E \Rightarrow_{rm} E * E$$

$$\Rightarrow_{rm} E * E + E$$

$$\Rightarrow_{rm} E * E + \text{id}_3$$

$$\Rightarrow_{rm} E * \text{id}_2 + \text{id}_3$$

$$\Rightarrow_{rm} \text{id}_1 * \text{id}_2 + \text{id}_3$$

$$E \Rightarrow_{rm} E + E$$

$$\Rightarrow_{rm} E + \text{id}_3$$

$$\Rightarrow_{rm} E * E + \text{id}_3$$

$$\Rightarrow_{rm} E * \text{id}_2 + \text{id}_3$$

$$\Rightarrow_{rm} \text{id}_1 * \text{id}_2 + \text{id}_3$$



# 例 句柄不唯一

$$E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$$

$$E \Rightarrow_{rm} E * E$$

$$\Rightarrow_{rm} E * E + E$$

$$\Rightarrow_{rm} E * E + \text{id}_3$$

$$\Rightarrow_{rm} E * \text{id}_2 + \text{id}_3$$

$$\Rightarrow_{rm} \text{id}_1 * \text{id}_2 + \text{id}_3$$

$$E \Rightarrow_{rm} E + E$$

$$\Rightarrow_{rm} E + \text{id}_3$$

$$\Rightarrow_{rm} E * E + \text{id}_3$$

$$\Rightarrow_{rm} E * \text{id}_2 + \text{id}_3$$

$$\Rightarrow_{rm} \text{id}_1 * \text{id}_2 + \text{id}_3$$

在右句型  $E * E + \text{id}_3$  中，句柄不唯一：  $\text{id}_3$ 、  $E * E$

\* 右句型：最右推导可得句型



中国科学技术大学  
University of Science and Technology of China

# 用栈实现移进-归约解析



# 用栈实现移进-归约解析

先通过“移进-归约解析器在解析输入串 $id_1 * id_2 + id_3$ 时的动作序列”来了解移进-归约解析的工作方式。

需要引入**栈**保存移进的文法符号

归约时需要从栈的顶部将形成句柄的文法符号串弹出，再将归约成的非终结符入栈

初始格局：

| 栈  | 输入  |
|----|-----|
| \$ | w\$ |

解析成功后的格局：

| 栈   | 输入 |
|-----|----|
| \$S | \$ |



# 移进-归约解析: $id_1 * id_2 + id_3$

| 栈  | 输 入                     | 动 作 |
|----|-------------------------|-----|
| \$ | $id_1 * id_2 + id_3$ \$ |     |
|    |                         |     |
|    |                         |     |
|    |                         |     |
|    |                         |     |
|    |                         |     |
|    |                         |     |
|    |                         |     |
|    |                         |     |
|    |                         |     |
|    |                         |     |



# 移进-归约解析: $id_1 * id_2 + id_3$

| 栈  | 输 入                     | 动 作 |
|----|-------------------------|-----|
| \$ | $id_1 * id_2 + id_3$ \$ | 移进  |
|    |                         |     |
|    |                         |     |
|    |                         |     |
|    |                         |     |
|    |                         |     |
|    |                         |     |
|    |                         |     |
|    |                         |     |
|    |                         |     |
|    |                         |     |



# 移进-归约解析: $id_1 * id_2 + id_3$

| 栈         | 输 入                     | 动 作 |
|-----------|-------------------------|-----|
| \$        | $id_1 * id_2 + id_3 \$$ | 移进  |
| \$ $id_1$ | $* id_2 + id_3 \$$      |     |
|           |                         |     |
|           |                         |     |
|           |                         |     |
|           |                         |     |
|           |                         |     |
|           |                         |     |
|           |                         |     |
|           |                         |     |
|           |                         |     |



# 移进-归约解析: $id_1 * id_2 + id_3$

| 栈         | 输 入                     | 动 作                     |
|-----------|-------------------------|-------------------------|
| \$        | $id_1 * id_2 + id_3 \$$ | 移进                      |
| \$ $id_1$ | $* id_2 + id_3 \$$      | 按 $E \rightarrow id$ 归约 |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |



# 移进-归约解析: $id_1 * id_2 + id_3$

| 栈           | 输 入                     | 动 作                     |
|-------------|-------------------------|-------------------------|
| \$          | $id_1 * id_2 + id_3 \$$ | 移进                      |
| \$ $id_1$   | $* id_2 + id_3 \$$      | 按 $E \rightarrow id$ 归约 |
| \$ <i>E</i> | $* id_2 + id_3 \$$      |                         |
|             |                         |                         |
|             |                         |                         |
|             |                         |                         |
|             |                         |                         |
|             |                         |                         |
|             |                         |                         |
|             |                         |                         |
|             |                         |                         |



# 移进-归约解析: $id_1 * id_2 + id_3$

| 栈         | 输 入                     | 动 作                     |
|-----------|-------------------------|-------------------------|
| \$        | $id_1 * id_2 + id_3 \$$ | 移进                      |
| \$ $id_1$ | $* id_2 + id_3 \$$      | 按 $E \rightarrow id$ 归约 |
| $\$E$     | $* id_2 + id_3 \$$      | 移进                      |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |



# 移进-归约解析: $id_1 * id_2 + id_3$

| 栈         | 输 入                     | 动 作                     |
|-----------|-------------------------|-------------------------|
| \$        | $id_1 * id_2 + id_3 \$$ | 移进                      |
| \$ $id_1$ | $* id_2 + id_3 \$$      | 按 $E \rightarrow id$ 归约 |
| $\$E$     | $* id_2 + id_3 \$$      | 移进                      |
| $\$E*$    | $id_2 + id_3 \$$        |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |



# 移进-归约解析: $id_1 * id_2 + id_3$

| 栈         | 输 入                     | 动 作                     |
|-----------|-------------------------|-------------------------|
| \$        | $id_1 * id_2 + id_3 \$$ | 移进                      |
| \$ $id_1$ | $* id_2 + id_3 \$$      | 按 $E \rightarrow id$ 归约 |
| $\$E$     | $* id_2 + id_3 \$$      | 移进                      |
| $\$E*$    | $id_2 + id_3 \$$        | 移进                      |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |



# 移进-归约解析: $id_1 * id_2 + id_3$

| 栈         | 输 入                     | 动 作                     |
|-----------|-------------------------|-------------------------|
| \$        | $id_1 * id_2 + id_3 \$$ | 移进                      |
| \$ $id_1$ | $* id_2 + id_3 \$$      | 按 $E \rightarrow id$ 归约 |
| $SE$      | $* id_2 + id_3 \$$      | 移进                      |
| $SE*$     | $id_2 + id_3 \$$        | 移进                      |
| $SE*id_2$ | $+ id_3 \$$             |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |
|           |                         |                         |



# 移进-归约解析: $id_1 * id_2 + id_3$

| 栈           | 输 入                     | 动 作                     |
|-------------|-------------------------|-------------------------|
| \$          | $id_1 * id_2 + id_3 \$$ | 移进                      |
| \$ $id_1$   | $* id_2 + id_3 \$$      | 按 $E \rightarrow id$ 归约 |
| \$ $E$      | $* id_2 + id_3 \$$      | 移进                      |
| \$ $E*$     | $id_2 + id_3 \$$        | 移进                      |
| \$ $E*id_2$ | $+ id_3 \$$             | 按 $E \rightarrow id$ 归约 |
|             |                         |                         |
|             |                         |                         |
|             |                         |                         |
|             |                         |                         |
|             |                         |                         |
|             |                         |                         |



# 移进-归约解析: $id_1 * id_2 + id_3$

| 栈                         | 输 入                     | 动 作                     |
|---------------------------|-------------------------|-------------------------|
| \$                        | $id_1 * id_2 + id_3 \$$ | 移进                      |
| \$ $id_1$                 | $* id_2 + id_3 \$$      | 按 $E \rightarrow id$ 归约 |
| $\$E$                     | $* id_2 + id_3 \$$      | 移进                      |
| $\$E*$                    | $id_2 + id_3 \$$        | 移进                      |
| $\$E*id_2$                | $+ id_3 \$$             | 按 $E \rightarrow id$ 归约 |
| $\$E*\textcolor{blue}{E}$ | $+ id_3 \$$             |                         |
|                           |                         |                         |
|                           |                         |                         |
|                           |                         |                         |
|                           |                         |                         |
|                           |                         |                         |



# 移进-归约解析: $id_1 * id_2 + id_3$

| 栈                                                                                                                                                                                                                                                                                                                                                                                            | 输 入                     | 动 作                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|-------------------------|
| \$                                                                                                                                                                                                                                                                                                                                                                                           | $id_1 * id_2 + id_3 \$$ | 移进                      |
| \$ $id_1$                                                                                                                                                                                                                                                                                                                                                                                    | $* id_2 + id_3 \$$      | 按 $E \rightarrow id$ 归约 |
| \$ $E$                                                                                                                                                                                                                                                                                                                                                                                       | $* id_2 + id_3 \$$      | 移进                      |
| \$ $E*$                                                                                                                                                                                                                                                                                                                                                                                      | $id_2 + id_3 \$$        | 移进                      |
| \$ $E*id_2$                                                                                                                                                                                                                                                                                                                                                                                  | $+ id_3 \$$             | 按 $E \rightarrow id$ 归约 |
| \$ $E*E$                                                                                                                                                                                                                                                                                                                                                                                     | $+ id_3 \$$             | 移进? 归约?                 |
| <div><div><math display="block">\begin{aligned} E &amp;\Rightarrow_{rm} E * E \\ &amp;\Rightarrow_{rm} E * E + E \\ &amp;\Rightarrow_{rm} E * E + id_3 \end{aligned}</math><div>移进</div></div><div><math display="block">\begin{aligned} E &amp;\Rightarrow_{rm} E + E \\ &amp;\Rightarrow_{rm} E + id_3 \\ &amp;\Rightarrow_{rm} E * E + id_3 \end{aligned}</math><div>归约</div></div></div> |                         |                         |
|                                                                                                                                                                                                                                                                                                                                                                                              |                         |                         |
|                                                                                                                                                                                                                                                                                                                                                                                              |                         |                         |



# 移进-归约解析: $id_1 * id_2 + id_3$

| 栈              | 输入                             | 动作                       |
|----------------|--------------------------------|--------------------------|
| \$             | $id_1 * id_2 + id_3 \$$        | 移进                       |
| \$ $id_1$      | $* id_2 + id_3 \$$             | 按 $E \rightarrow id$ 归约  |
| \$E            | $* id_2 + id_3 \$$             | 移进                       |
| \$E*           | $id_2 + id_3 \$$               | 移进                       |
| \$ E* $id_2$   | $+ id_3 \$$                    | 按 $E \rightarrow id$ 归约  |
| <b>\$ E*E</b>  | <b><math>+ id_3 \\$</math></b> | <b>移进</b>                |
| \$ E*E+        | $id_3 \$$                      | 移进                       |
| \$ E*E+ $id_3$ | \$                             | 按 $E \rightarrow id$ 归约  |
| \$ E*E+E       | \$                             | 按 $E \rightarrow E+E$ 归约 |
| \$ E*E         | \$                             | 按 $E \rightarrow E*E$ 归约 |
| <b>\$E</b>     | <b>\$</b>                      | <b>接受</b>                |



# 自下而上解析

(移进-归约解析)

- 归约(最右推导的逆过程)
- 句柄(可归约串),可能不唯一
- 冲突: 移进-归约、归约-归约



# 移进-归约解析需要解决的一些问题

- 怎么确定右句型中要归约的子串(即句柄)
  - 句柄总是出现在栈顶 (长度大于等于0的语法符号串)
- 面临**移进-归约冲突**时, 如何决策是选择移进还是归约?
- 面临**归约-归约冲突**时, 不知该选择哪一个产生式归约?
  - 栈顶形成不同的句柄, 或
  - 句柄是不止一个产生式的右部



# 移进-归约解析的冲突

## □ 移进-归约冲突( shift/reduce conflict)

例

$stmt \rightarrow$  **if**  $expr$  **then**  $stmt$   
          | **if**  $expr$  **then**  $stmt$  **else**  $stmt$   
          | **other**

如果移进-归约解析器处于格局(configuration)

栈

输入

... **if**  $expr$  **then**  $stmt$

**else** ... \$

**一般地，优先移进**  
也满足else的  
就近匹配原则



# 移进-归约解析的冲突

## □ 归约-归约冲突(reduce/reduce conflict)

$stmt \rightarrow id (parameter\_list) \mid expr = expr$        $id(...)$ 是过程调用

$parameter\_list \rightarrow parameter\_list, parameter \mid parameter$

$parameter \rightarrow id$

$expr \rightarrow id (expr\_list) \mid id$        $id(...)$ 也表示数组元素的引用

$expr\_list \rightarrow expr\_list, expr \mid expr$

由 $A(I, J)$ 开始的语句

栈

... id ( id

归约成 $expr$ 还是 $parameter$ ?

输入

, id )...

**方法1**  
一般用位于前面的产生式进行归约



# 移进-归约解析的冲突

## □ 归约-归约冲突(reduce/reduce conflict)

$stmt \rightarrow \text{id}(parameter\_list) \mid expr = expr$

$parameter\_list \rightarrow parameter\_list, parameter \mid parameter$

$parameter \rightarrow id$

$expr \rightarrow id(expr\_list) \mid id$        $id(...)$ 也表示数组元素的引用

$expr\_list \rightarrow expr\_list, expr \mid expr$

由 $A(I, J)$ 开始的语句(词法分析查符号表, 区分第一个id)

栈

... **procid**( id

输入

, id )...

■ 需要修改上面的文法

**方法2**  
改写文法, 区分  
id是否是procid